Nakijk 02_02 — V4

NAKIJK V4 · Hoofdstuk 02 · Volledige CRUD · bij Oefening_02_02.md · 16 oktober 2026

Nagekeken: `ProgrammingAdvancedOefeningen` → `Models/Ventilator.cs` (11 oktober) en `Controllers/VentilatorController.cs` (16 oktober, 18:38).

> **OPMERKING**
>
> Nagekeken: punt 1 tot en met 7. Punt 8 maak je later en blijft hier buiten.
>
> Nieuw in deze versie: ik heb elk verzoek dat het nakijkscript voor punt 1 tot 7 stuurt, op papier nagelopen tegen je code. Zo zie je per test of hij zou slagen.
>
> Ik kon je API niet starten, want op deze werkruimte staat geen .NET. Ik heb je code gelezen en elk verzoek op papier nagelopen.

In één oogopslag

Punt 4 is af: de drie controles werken, ze eindigen alle drie met `return` en ze staan vóór het toevoegen. Je hebt ze ook al in je PUT gezet, op de juiste plaats. Van de 18 testen van het nakijkscript voor punt 1 tot 7 zouden er **16 slagen**.

Wat nog fout loopt, zit in de DELETE: een onbekend Id geeft nog altijd 204 (A1). Daarnaast wijken twee berichten af van de opgave (A2).

Punt	Wat de opgave vraagt	Status
1	het model `Ventilator`	✓ in orde
2	de controller, met ventilatoren die bewaard blijven	✓ in orde
3	`GET api/ventilator` en `GET api/ventilator/{id}`	✓ in orde, het bericht heeft nu zijn punt
4	`POST api/ventilator` met 201 en drie controles	✓ in orde, zie wel B1
5	`PUT api/ventilator/{id}` met 204, 404 en 400	½ zie A2
6	`DELETE api/ventilator/{id}` met 204 en 404	✗ zie A1 en A2
7	de twee testen	½ 92 mm geeft 400 ✓, een tweede DELETE geeft nog 204 ✗
8	`PUT api/ventilator/vervang/{id}`	niet nagekeken

Wat het nakijkscript zou zeggen

Dit zijn de verzoeken uit het nakijkscript voor punt 1 tot 7, in de volgorde waarin het script ze stuurt. Het script vergelijkt de statuscode en het bericht letterlijk. Bij de Location-header tellen hoofdletters niet mee.

Test	Verzoek	Het script verwacht	Je API antwoordt	
3.1	`GET api/ventilator`	200, elke ventilator met de vijf velden	200, vier ventilatoren	✓
4.1	`POST` Noctua, 120 mm, 24.95	201, adres eindigt op */api/ventilator/5*	201, Id 5, adres */api/Ventilator/5*	✓
4.2	`GET api/ventilator/5`	200 met de Noctua	200	✓
4.3	`POST` met `"id": 999`	201, met een Id dat de API zelf kiest	201, Id 6	✓
4.4	`GET api/ventilator/999`	404 met *We vonden geen ventilator met Id 999.*	hetzelfde	✓
4.5	`POST` met `"merk": ""`	400 met *Een ventilator moet een merk hebben.*	hetzelfde	✓
4.6	`POST` met 92 mm	400 met *We verkopen enkel ventilatoren van 120 of 140 mm.*	hetzelfde	✓
4.7	`POST` met prijs 0	400 met *De prijs moet groter zijn dan 0.*	hetzelfde	✓
7.1	`GET api/ventilator`	geen ventilator van 92 mm in de lijst	er staat er geen in	✓
5.1	`PUT api/ventilator/5` naar 140 mm, 29.95	204	204	✓
5.2	`GET api/ventilator/5`	200 met de nieuwe gegevens	200, 140 mm, 29.95	✓
5.3	`PUT api/ventilator/999`	404 met *Kan geen ventilator bijwerken met Id 999, omdat deze niet bestaat.*	404 met *Kan geen Ventilator bijwerken …*	✗ A2
5.4	`PUT` met `"merk": ""`	400	400	✓
5.5	`PUT` met 92 mm	400	400	✓
5.6	`PUT` met prijs 0	400	400	✓
6.1	`DELETE api/ventilator/5`	204	204	✓
6.2	nog eens `DELETE api/ventilator/5`	404 met *Kan ventilator met Id 5 niet verwijderen, omdat deze niet gevonden is.*	204	✗ A1 en A2
6.3	`GET api/ventilator/5`	404 met *We vonden geen ventilator met Id 5.*	hetzelfde	✓

Het script test punt 8 in een tweede reeks. Die testen zullen falen zolang punt 8 er niet staat. Dat is hier geen fout.

A. Wat een verkeerd antwoord geeft

A1. Een DELETE op een onbekend Id geeft nog altijd 204

Waar: `DeleteVentilator`, regel 98 tot 103. Dit stond ook in V3, bij A2.

Verzoek	Je API antwoordt nu	De opgave vraagt
`DELETE api/ventilator/2`	204	✓
nog eens `DELETE api/ventilator/2`	204	404 met *Kan ventilator met Id 2 niet verwijderen, omdat deze niet gevonden is.*
`DELETE api/ventilator/99`	204	404 met het bericht hierboven

Waarom. Op regel 100 maak je een 404 met `NotFound(...)`, maar je stuurt ze niet terug. Je methode gaat dan verder naar regel 103 en roept `Remove` op met `null`. Daar crasht niets, maar er wordt ook niets verwijderd, en regel 105 geeft een 204.

Je hebt in punt 4 precies dit opgelost. Leg regel 100 naast regel 42 in je POST, of naast regel 67 in je PUT. Wat staat daar vóór de helper, dat op regel 100 ontbreekt?

De compiler waarschuwt je hiervoor op regel 103: **CS8604**, Possible null reference argument for parameter 'item' in 'bool List<Ventilator>.Remove(Ventilator item)'. Zodra je `if` de methode stopt, weet de compiler dat `ventilator` op regel 103 nooit `null` is, en verdwijnt de waarschuwing vanzelf. Zo weet je meteen dat het opgelost is.

A2. Twee berichten wijken af

Het script vergelijkt elk bericht letterlijk. Eén letter verschil is al een fout.

Waar	Jouw bericht	De opgave
regel 67, PUT	Kan geen **V**entilator bijwerken met Id {id}, omdat deze niet bestaat.	Kan geen **v**entilator bijwerken met Id {id}, omdat deze niet bestaat.
regel 100, DELETE	…, omdat deze niet **is gevonden**.	…, omdat deze niet **gevonden is**.

Let op bij de DELETE: test 6.2 slaagt pas als A1 **én** dit bericht in orde zijn.

Richting. Kopieer beide berichten uit de opgave en plak ze in je code.

B. Werkt, maar kan beter

B1. Een merk met enkel spaties komt erdoor

Waar: regel 40 in je POST en regel 70 in je PUT.

Je controle kijkt of het merk gelijk is aan een lege tekst. Een merk van drie spaties is geen lege tekst, en komt er dus door:

Verzoek	Je API antwoordt nu	Beter
`POST` met `"merk": "   "`	201, en een ventilator zonder zichtbaar merk staat in de lijst	400 met *Een ventilator moet een merk hebben.*

Het nakijkscript test dit niet, en de opgave zegt alleen "leeg". Daarom staat het hier en niet bij A. Maar wie enkel spaties typt, heeft niets ingevuld.

Richting. Je leerboek, hoofdstuk 1, tabel *Hoe lang is het, is het leeg?*. Daar staat één methode die een lege tekst én een tekst met enkel spaties in één keer nagaat. Zie ook V3, stap 3.

B2. Dezelfde drie controles staan twee keer

Waar: regel 40 tot 51 in je POST, regel 70 tot 81 in je PUT.

Beide blokken zijn gelijk, en dat werkt. Maar B1 toont meteen het nadeel: je moet dezelfde verbetering nu op twee plaatsen doen, en vergeet je er één, dan gedraagt je POST zich anders dan je PUT. In punt 8 heb je dezelfde controles een derde keer nodig.

Richting. Zie V3, stap 8. Zet de controles op één plaats in je controller, zodat de POST, de PUT en straks de vervanging ze kunnen gebruiken. Denk na wat die plaats aan je POST moet vertellen, zodat je POST weet óf hij moet weigeren, en met welk bericht. Doe dit vóór je aan punt 8 begint: dan schrijf je de vervanging meteen op de nieuwe manier.

C. Opkuis, zonder invloed op het antwoord

Waar	Wat	Richting
regel 37 en 38	Je bepaalt het nieuwe Id nog vóór de controles. Het doet geen kwaad, want een geweigerde ventilator komt niet in de lijst. Maar je geeft een nummer aan iets dat je misschien weigert.	Zet het Id vlak voor het toevoegen.
regel 44 tot 57, en 74 tot 90	Na elke `return` volgt een `else`. Dat werkt, maar het is niet nodig: na een `return` gaat de methode toch niet verder. Zonder `else` staat wat er gebeurt als alles in orde is, onderaan en niet ingesprongen.	Kijk hoe je het in `GetVentilatorById` doet: een `if` met `return`, en daaronder gewoon verder. Kies één manier voor je hele controller.
regel 24 en 37, tegenover 63 en 96	De parameter in je lambda heet bij de GET en de POST `fan`, bij de PUT en de DELETE `ventilator`.	Kies één schrijfwijze. De les gebruikt een korte letter, zoals `x`.
regel 96	De variabele `ventilator` en de parameter in de lambda heten hetzelfde, op dezelfde regel. Het compileert, maar bij het lezen zie je niet welke bedoeld is.	Geef de parameter in de lambda een andere naam.

D. Wat goed is, en waarom

- **Elke controle eindigt met `return`.** Een geweigerde ventilator komt daardoor nooit bij `Add` of bij het overnemen van de gegevens. Dat was de kern van punt 4, en je hebt het in de POST én in de PUT juist.
- **De volgorde klopt.** In je POST controleer je vóór je toevoegt. In je PUT zoek je eerst, geef je een 404 als de ventilator niet bestaat, controleer je daarna, en neem je pas dan de gegevens over. Wat je afkeurt, laat zo geen spoor na in de lijst.
- **120 of 140, met `&&`.** Een afmeting is fout als ze geen 120 **en** geen 140 is. Met `||` had je API elke ventilator geweigerd. Je bent die valkuil uit de weg gegaan.
- **De prijs met `<=`.** Zo wordt 0 zelf ook geweigerd, zoals de opgave vraagt.
- **De berichten van de drie controles** zijn letterlijk die van de opgave, en het bericht van je GET heeft nu zijn punt.
- **De volgorde van je controles** is die van de opgave: merk, afmeting, prijs. Is er meer dan één veld fout, dan krijgt de client het bericht van de eerste. Testen 4.5 tot 4.7 en 5.4 tot 5.6 slagen allemaal.
- **Je PUT neemt het Id niet over** uit de body, en geeft 204 met `NoContent`. Het Id komt uit de URL, en de client kan het zo niet wijzigen.
- **Je lijst is `private static`** en blijft bestaan tussen twee verzoeken. Daardoor slagen testen 4.2 en 5.2.

Compilerwaarschuwingen

Voor zover ik kan nagaan door te lezen, geeft je code **één** waarschuwing: **CS8604** op regel 103, bij A1. Ze verdwijnt zodra je `if` in de DELETE de methode stopt.

In Visual Studio zie je ze in de Error List, bij **Warnings**.

Zelf nakijken

Doe na het oplossen van A1 en A2 deze verzoeken. Een antwoord klopt pas als de statuscode én het bericht letterlijk overeenkomen met de opgave.

Verzoek	Wat je moet zien	Hoort bij
`PUT api/ventilator/99` met geldige gegevens	404 met *Kan geen ventilator bijwerken met Id 99, omdat deze niet bestaat.*	A2
`DELETE api/ventilator/2`	204	punt 6
nog eens `DELETE api/ventilator/2`	404 met *Kan ventilator met Id 2 niet verwijderen, omdat deze niet gevonden is.*	A1 en A2
`DELETE api/ventilator/99`	404 met hetzelfde bericht, met Id 99	A1
bouwen met Ctrl+Shift+B	geen waarschuwing CS8604 meer	A1
`POST` met `"merk": "  "`	400 met *Een ventilator moet een merk hebben.*	B1
`PUT api/ventilator/1` met `"merk": "  "`	400	B1

Met het nakijkscript, als je API in Visual Studio gesloten is:

```
.\Nakijk.ps1 -Oefening 02_02 -Project
"...\ProgrammingAdvancedOefeningen\ProgrammingAdvancedOefeningen"
```

Na A1 en A2 zou de eerste reeks volledig moeten slagen: 18 op 18. De tweede reeks gaat over punt 8 en faalt tot je dat punt gemaakt hebt.

Vragen die je je misschien stelt

Waarom slaagt test 6.1 wel, als mijn DELETE niet klopt? Bij 6.1 bestaat de ventilator nog. Dan klopt je DELETE volledig: hij vindt de ventilator, verwijdert hem en geeft 204. De fout zit alleen in het geval dat de ventilator er niet is, en dat test 6.2.

Waarom slaagt 6.3, als de DELETE in 6.2 niet klopt? 6.3 vraagt de ventilator op met een GET, en die zit in je GET-methode, niet in je DELETE. De ventilator is in 6.1 verwijderd, en je GET geeft dan de juiste 404 met het juiste bericht.

Moet ik de `else` echt weghalen? Nee. Het werkt zoals het staat, en het nakijkscript ziet geen verschil. Het is een kwestie van leesbaarheid: met een `if` en `return` zonder `else` zie je in één oogopslag wat er gebeurt als alles in orde is. Wat je ook kiest, doe het overal op dezelfde manier.

Waarom staat de controle op spaties niet bij A, als ik ze toch moet oplossen? Bij A staat wat een ander antwoord geeft dan de opgave of de testen vragen. De opgave zegt "leeg", en een tekst met spaties is strikt genomen niet leeg. Het is wel wat een echte gebruiker zou verwachten, en daarom staat het bij B: het werkt, maar het kan beter.